

LeetCode 1652: Defuse the Bomb

Amazon

Bloomberg

Complete Technical Interview Solution & Analysis

Author: Arpit Raj

Institution:

LNMIIT Jaipur

Date: June 29, 2026

Topic:

Circular Arrays & Sliding Window

Problem Breakdown & Core Logic

The problem requires us to decrypt a circular array based on a given key k . The transformation rules are explicitly defined as follows:

Core Operational Rules:

- **Rule 1 (If $k > 0$):** Replace every i -th member with the sum of the **next k members** (Circular logic applies).
- **Rule 2 (If $k < 0$):** Replace every i -th member with the sum of the **previous k members** (Circular logic applies).
- **Rule 3 (If $k = 0$):** Replace all members in the array with 0 .

Key Insight: Regardless of the case, the goal is to efficiently maintain and evaluate a **fixed-size window sum in a circular array layout** without redundant recalculations.

Approach 1: Optimized Sliding Window

Instead of re-summing k elements for every single index, we can initialize a sliding window once and shift it dynamically. When the window slides right, we subtract the outgoing element and add the incoming element.

Time Complexity: $O(N)$

Space Complexity: $O(1)$ [Auxiliary]

Execution Steps:

- 1 **Case A ($k > 0$):** The initial window elements are positioned from index 1 to k .
- 2 **Case B ($k < 0$):** The window starts before index 0 . We invert $k = -k$ and position the initial window from **start** = $n - k$ to **end** = $n - 1$.

3 **Window Slide:** Set *ans[i] = sum*, subtract *code[start % n]*, advance boundaries, and add *code[end % n]*.

C++ IMPLEMENTATION - SLIDING WINDOW

```
class Solution {
public:
    vector<int> decrypt(vector<int>& code, int k) {
        int n=code.size();
        vector<int> ans(n,0);
        if(k==0) return ans;
        int start;
        int end;
        if(k>0){

            start=1;
            end=k;

        }

        else if(k<0){
            k=-k;
            end=n-1;
            start=n-k;
        }

        int sum=0;
        for(int i=start;i<=end;i++){
            sum+=code[i%n];
        }
        for(int i=0;i<n;i++){
            ans[i]=sum;
            sum-=code[start%n];
            start++;
            end++;
            sum+=code[end%n];
        }

        return ans;
    }
};
```

Approach 2: Prefix Sum on a Doubled Array

To seamlessly handle the circular nature of the array without using complex modular arithmetic blocks inside inner loops, we can simulate circular continuity by **doubling the array** (concatenating it with itself).

For instance, an array $[5, 7, 1, 4]$ is extended to $[5, 7, 1, 4, 5, 7, 1, 4]$. A prefix sum array built on top of this doubled structure gives us any subsegment sum in $O(1)$ time.

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Execution Steps:

- ❶ **Array Ingestion:** Create an array of size $2 \times n$ and mirror the elements.
- ❷ **Prefix Sum Construction:** Populate a standard prefix array such that $prefix[i + 1] = prefix[i] + extended[i]$.
- ❸ **Case Processing:**
 - If $k > 0$: Define range bounds as $left = i + 1$ and $right = i + k$.
 - If $k < 0$: Shift operations into the second copy space by letting $idx = i + n$, setting $left = idx - k$ and $right = idx - 1$.

```
vector<int> extended(2 * n);

    for(int i = 0; i < 2 * n; i++)
    {
        extended[i] = code[i % n];
    }

    // -----
    // Step 2: Build prefix sum
    // -----
    vector<int> prefix(2 * n + 1, 0);

    for(int i = 0; i < 2 * n; i++)
    {
        prefix[i + 1] =
            prefix[i] + extended[i];
    }

    // -----
    // Case 1 : k > 0
    // -----
    if(k > 0)
    {
        for(int i = 0; i < n; i++)
        {
            int left = i + 1;
            int right = i + k;

            ans[i] =
                prefix[right + 1] - prefix[left];
        }
    }

    // -----
    // Case 2 : k < 0
    // -----
    else
    {
        k = -k;

        for(int i = 0; i < n; i++)
        {
            // Shift into second copy
            int idx = i + n;

            int left = idx - k;
            int right = idx - 1;

            ans[i] =
```

```
        prefix[right + 1] - prefix[left];
    }
}

return ans;
}
```